READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))


all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")
```

```
print(f"pneumon : {len(all_pneumonia_files)}")
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

```
train_pneumonia_samples = random_select_files(all_pneumonia_files, 1181)
test_pneumonia_samples = random_select_files([f for f in all_pneumonia_files if f not in train_pneumonia_samples], 394)

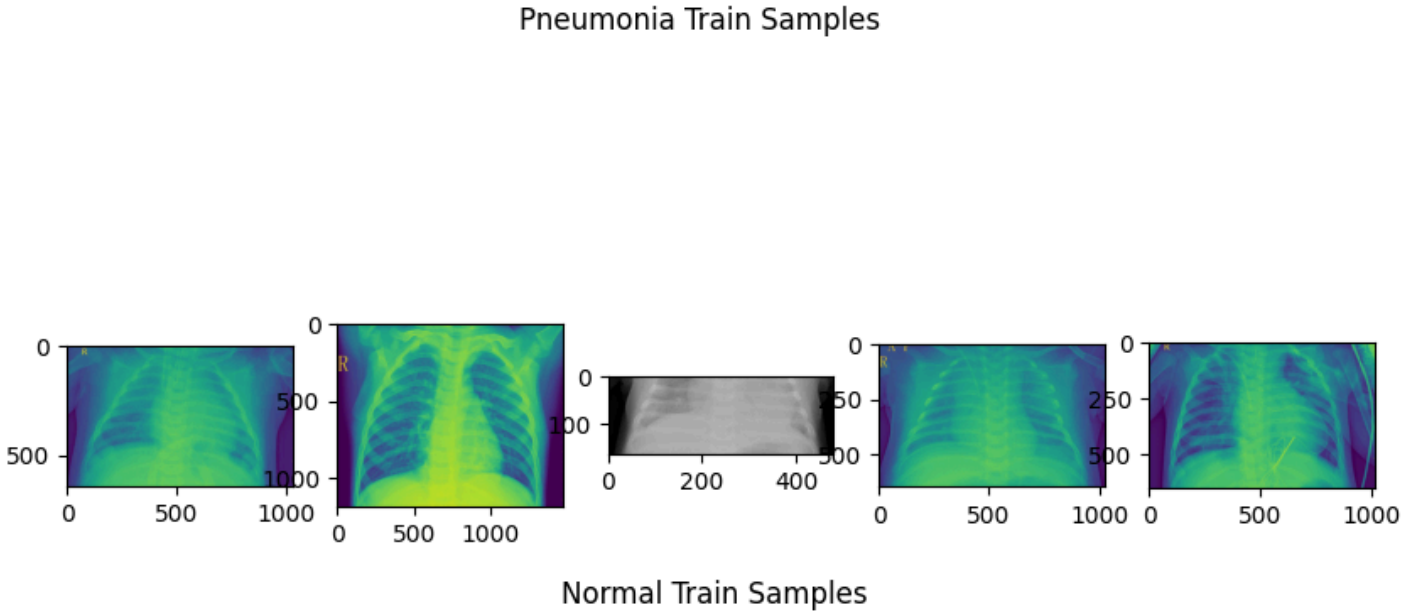
train_normal_samples = random_select_files(all_normal_files, 1181)
test_normal_samples = random_select_files([f for f in all_normal_files if f not in train_normal_samples], 394)

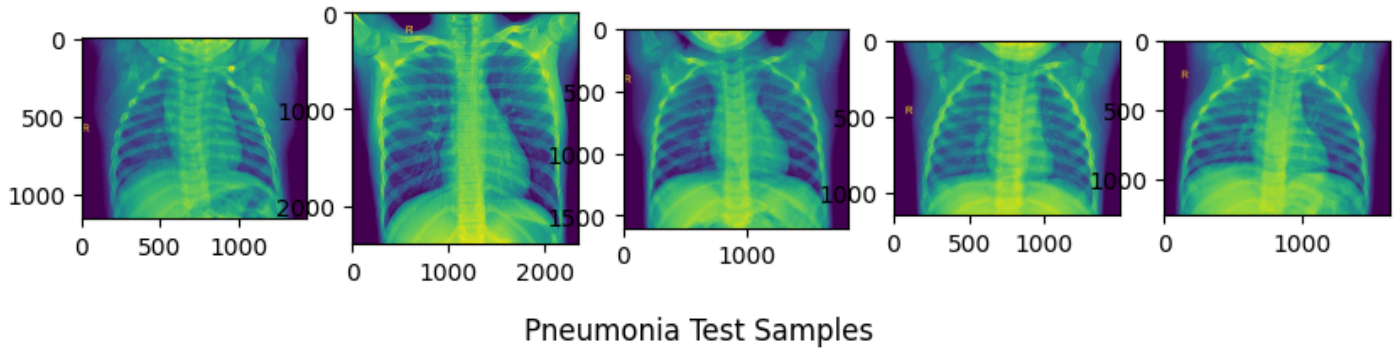
#train_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in train_pneumonia_files]
#test_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in test_pneumonia_files]
#train_normal_samples = [os.path.join(train_normal_dir, file) for file in train_normal_files]
#test_normal_samples = [os.path.join(train_normal_dir, file) for file in test_normal_files]

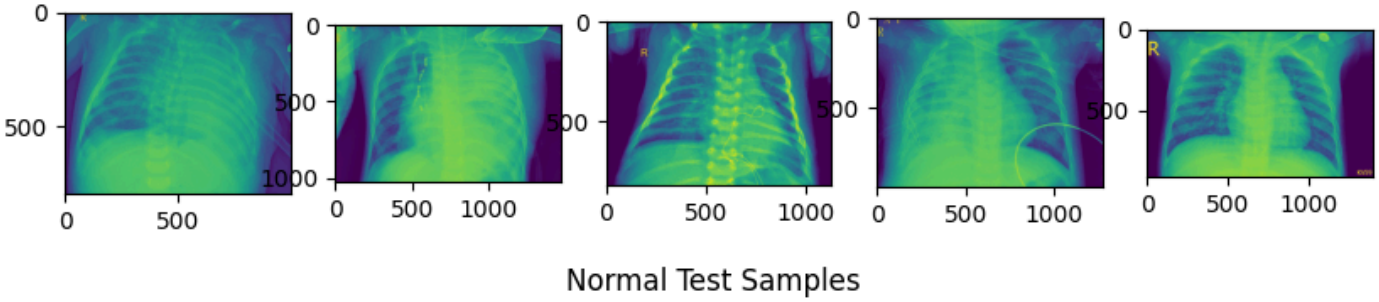

from PIL import Image
import matplotlib.pyplot as plt

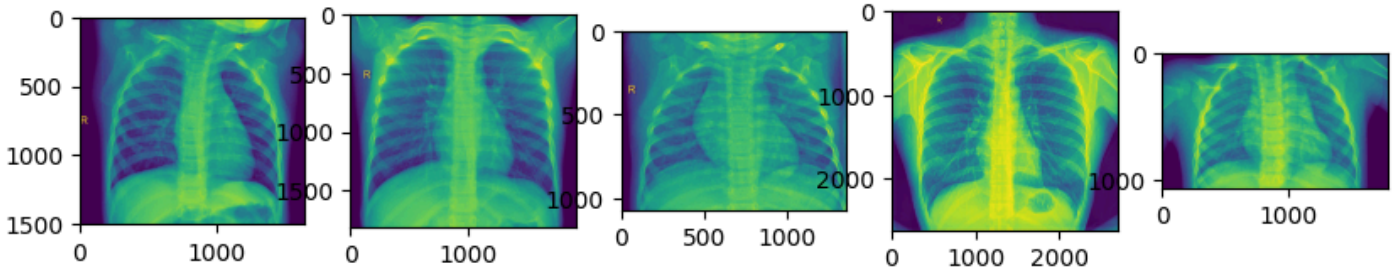
def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
        img = Image.open(sample_files[i])
        plt.imshow(img)
    plt.show()

show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```


Pneumonia Train Samples
Normal Train Samples


Pneumonia Test Samples


Normal Test Samples



RESIZE

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

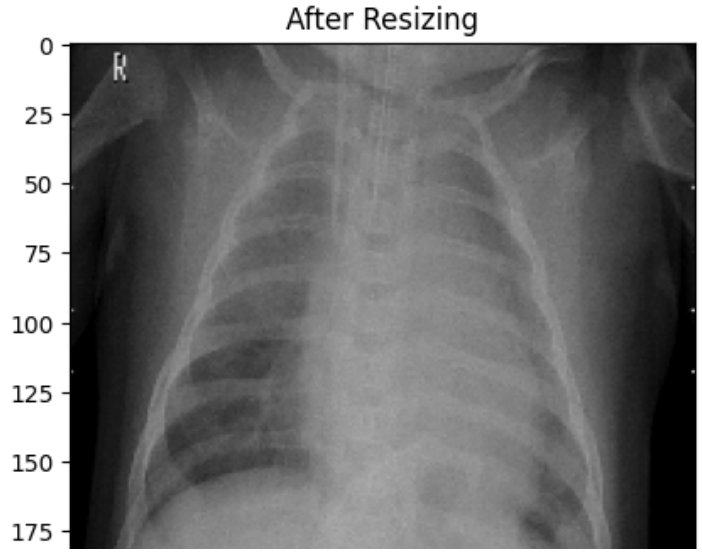
def load_images(file_paths, target_size=(224, 224)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

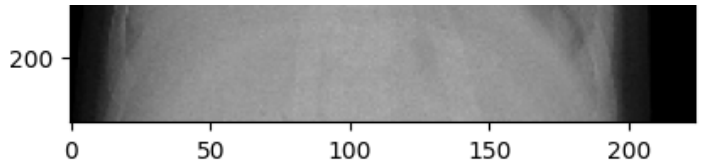
train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)


show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```




```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.1176
Text(0.5, 1.0, 'After Resizing')
```





GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```

```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
        glcm_features_list.append(features)
    return glcm_features_list

train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
train_normal_glcm = extract_glcm_features(train_normal_images)
test_normal_glcm = extract_glcm_features(test_normal_images)

print("GLCM features for the first pneumonia training image:")
train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(464.346497995082),
 'contrast_std': np.float64(232.5454150556531),
 'contrast_var': np.float64(54077.37006340597),
 'dissimilarity_mean': np.float64(7.345492127609162),
 'dissimilarity_std': np.float64(1.9541222365747073),
 'dissimilarity_var': np.float64(3.818593715475736),
 'homogeneity_mean': np.float64(0.19131759226687206),
 'homogeneity_std': np.float64(0.03236271913395924),
 'homogeneity_var': np.float64(0.0010473455897435314),
 'energy_mean': np.float64(0.018500767295068086),
 'energy_std': np.float64(0.0014792844211961493),
 'energy_var': np.float64(2.1882823987936264e-06),
 'correlation_mean': np.float64(0.9100578343973263),
 'correlation_std': np.float64(0.04636346344645034),
 'correlation_var': np.float64(0.0021495707427503373),
 'ASM_mean': np.float64(0.0003444666729050545),
 'ASM_std': np.float64(5.489525097919526e-05),
 'ASM_var': np.float64(3.0134885800688384e-09),
 'entropy': np.float64(144.61865836334988)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'
```

```
#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'

#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)
```

```
# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(32, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=50, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=100,
```

```
        batch_size=32,
        validation_split=0.3,
        callbacks=callbacks,
        class_weight=class_weights,
    )
```

```
r/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Whe
uper().__init__(activity_regularizer=activity_regularizer, **kwargs)
el: "sequential_5"
```

ayer (type)	Output Shape	Param #
ense_20 (Dense)	(None, 128)	2,560
atch_normalization_15 BatchNormalization)	(None, 128)	512
ropout_15 (Dropout)	(None, 128)	0
ense_21 (Dense)	(None, 64)	8,256
atch_normalization_16 BatchNormalization)	(None, 64)	256
ropout_16 (Dropout)	(None, 64)	0
ense_22 (Dense)	(None, 32)	2,080
atch_normalization_17 BatchNormalization)	(None, 32)	128
ropout_17 (Dropout)	(None, 32)	0
ense_23 (Dense)	(None, 1)	33

```
tal params: 13,825 (54.00 KB)
ainable params: 13,377 (52.25 KB)
n-trainable params: 448 (1.75 KB)
ch 1/100
52 ━━━━━━━━━━━━━━━━━━━━ 6s 17ms/step - accuracy: 0.6293 - auc_5: 0.6781 - loss: 0.7255 - val_accuracy: 0.5825 - val_auc_5: 0.0000e+00 - val_loss: 0.70
ch 2/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7958 - auc_5: 0.8814 - loss: 0.4763 - val_accuracy: 0.5882 - val_auc_5: 0.0000e+00 - val_loss: 0.725
ch 3/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8319 - auc_5: 0.9131 - loss: 0.4046 - val_accuracy: 0.6502 - val_auc_5: 0.0000e+00 - val_loss: 0.684
ch 4/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8284 - auc_5: 0.9010 - loss: 0.4340 - val_accuracy: 0.7870 - val_auc_5: 0.0000e+00 - val_loss: 0.555
ch 5/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8491 - auc_5: 0.9237 - loss: 0.3800 - val_accuracy: 0.8096 - val_auc_5: 0.0000e+00 - val_loss: 0.535
ch 6/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8569 - auc_5: 0.9201 - loss: 0.3795 - val_accuracy: 0.8124 - val_auc_5: 0.0000e+00 - val_loss: 0.528
ch 7/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8552 - auc_5: 0.9321 - loss: 0.3522 - val_accuracy: 0.8688 - val_auc_5: 0.0000e+00 - val_loss: 0.430
ch 8/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8734 - auc_5: 0.9164 - loss: 0.3788 - val_accuracy: 0.8872 - val_auc_5: 0.0000e+00 - val_loss: 0.383
ch 9/100
```

```
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8615 - auc_5: 0.9245 - loss: 0.3710 - val_accuracy: 0.8914 - val_auc_5: 0.0000e+00 - val_loss: 0.360
ch 10/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8772 - auc_5: 0.9384 - loss: 0.3319 - val_accuracy: 0.8914 - val_auc_5: 0.0000e+00 - val_loss: 0.352
ch 11/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8667 - auc_5: 0.9250 - loss: 0.3712 - val_accuracy: 0.8843 - val_auc_5: 0.0000e+00 - val_loss: 0.340
ch 12/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8791 - auc_5: 0.9359 - loss: 0.3365 - val_accuracy: 0.8956 - val_auc_5: 0.0000e+00 - val_loss: 0.336
ch 13/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8741 - auc_5: 0.9320 - loss: 0.3465 - val_accuracy: 0.9041 - val_auc_5: 0.0000e+00 - val_loss: 0.310
ch 14/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8667 - auc_5: 0.9401 - loss: 0.3257 - val_accuracy: 0.9013 - val_auc_5: 0.0000e+00 - val_loss: 0.305
ch 15/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8943 - auc_5: 0.9465 - loss: 0.3172 - val_accuracy: 0.9013 - val_auc_5: 0.0000e+00 - val_loss: 0.323
ch 16/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8871 - auc_5: 0.9469 - loss: 0.3100 - val_accuracy: 0.8956 - val_auc_5: 0.0000e+00 - val_loss: 0.334
ch 17/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8756 - auc_5: 0.9397 - loss: 0.3212 - val_accuracy: 0.8984 - val_auc_5: 0.0000e+00 - val_loss: 0.325
ch 18/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8931 - auc_5: 0.9453 - loss: 0.3065 - val_accuracy: 0.9069 - val_auc_5: 0.0000e+00 - val_loss: 0.28
ch 19/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8790 - auc_5: 0.9428 - loss: 0.3204 - val_accuracy: 0.9182 - val_auc_5: 0.0000e+00 - val_loss: 0.27
ch 20/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.8770 - auc_5: 0.9423 - loss: 0.3199 - val_accuracy: 0.9027 - val_auc_5: 0.0000e+00 - val_loss: 0.32
ch 21/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.8768 - auc_5: 0.9428 - loss: 0.3201 - val_accuracy: 0.8801 - val_auc_5: 0.0000e+00 - val_loss: 0.36
ch 22/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8737 - auc_5: 0.9347 - loss: 0.3320 - val_accuracy: 0.9168 - val_auc_5: 0.0000e+00 - val_loss: 0.297
ch 23/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8885 - auc_5: 0.9555 - loss: 0.2853 - val_accuracy: 0.8914 - val_auc_5: 0.0000e+00 - val_loss: 0.340
ch 24/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8941 - auc_5: 0.9577 - loss: 0.2791 - val_accuracy: 0.9097 - val_auc_5: 0.0000e+00 - val_loss: 0.303
ch 25/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8709 - auc_5: 0.9411 - loss: 0.3307 - val_accuracy: 0.9111 - val_auc_5: 0.0000e+00 - val_loss: 0.291
ch 26/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8936 - auc_5: 0.9492 - loss: 0.3034 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.286
ch 27/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8858 - auc_5: 0.9468 - loss: 0.3076 - val_accuracy: 0.8942 - val_auc_5: 0.0000e+00 - val_loss: 0.319
ch 28/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8995 - auc_5: 0.9522 - loss: 0.2957 - val_accuracy: 0.9140 - val_auc_5: 0.0000e+00 - val_loss: 0.291
ch 29/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9022 - auc_5: 0.9559 - loss: 0.2797 - val_accuracy: 0.9140 - val_auc_5: 0.0000e+00 - val_loss: 0.290
ch 30/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8889 - auc_5: 0.9546 - loss: 0.2859 - val_accuracy: 0.8942 - val_auc_5: 0.0000e+00 - val_loss: 0.334
ch 31/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8758 - auc_5: 0.9435 - loss: 0.3167 - val_accuracy: 0.8970 - val_auc_5: 0.0000e+00 - val_loss: 0.313
ch 32/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8919 - auc_5: 0.9540 - loss: 0.2869 - val_accuracy: 0.9041 - val_auc_5: 0.0000e+00 - val_loss: 0.316
ch 33/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8979 - auc_5: 0.9598 - loss: 0.2707 - val_accuracy: 0.8942 - val_auc_5: 0.0000e+00 - val_loss: 0.324
```

```
ch 34/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8908 - auc_5: 0.9546 - loss: 0.2903 - val_accuracy: 0.9182 - val_auc_5: 0.0000e+00 - val_loss: 0.279
ch 35/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8931 - auc_5: 0.9546 - loss: 0.2863 - val_accuracy: 0.9097 - val_auc_5: 0.0000e+00 - val_loss: 0.299
ch 36/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9036 - auc_5: 0.9621 - loss: 0.2629 - val_accuracy: 0.9154 - val_auc_5: 0.0000e+00 - val_loss: 0.280
ch 37/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8911 - auc_5: 0.9494 - loss: 0.3004 - val_accuracy: 0.8984 - val_auc_5: 0.0000e+00 - val_loss: 0.313
ch 38/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8981 - auc_5: 0.9593 - loss: 0.2690 - val_accuracy: 0.9027 - val_auc_5: 0.0000e+00 - val_loss: 0.296
ch 39/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9171 - auc_5: 0.9666 - loss: 0.2467 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.296
ch 40/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8839 - auc_5: 0.9484 - loss: 0.3067 - val_accuracy: 0.9252 - val_auc_5: 0.0000e+00 - val_loss: 0.257
ch 41/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.8956 - auc_5: 0.9620 - loss: 0.2632 - val_accuracy: 0.9013 - val_auc_5: 0.0000e+00 - val_loss: 0.318
ch 42/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9071 - auc_5: 0.9564 - loss: 0.2770 - val_accuracy: 0.8872 - val_auc_5: 0.0000e+00 - val_loss: 0.346
ch 43/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.8988 - auc_5: 0.9575 - loss: 0.2771 - val_accuracy: 0.8984 - val_auc_5: 0.0000e+00 - val_loss: 0.313
ch 44/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9112 - auc_5: 0.9585 - loss: 0.2695 - val_accuracy: 0.9210 - val_auc_5: 0.0000e+00 - val_loss: 0.274
ch 45/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8951 - auc_5: 0.9599 - loss: 0.2727 - val_accuracy: 0.9041 - val_auc_5: 0.0000e+00 - val_loss: 0.296
ch 46/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8918 - auc_5: 0.9520 - loss: 0.2899 - val_accuracy: 0.9210 - val_auc_5: 0.0000e+00 - val_loss: 0.267
ch 47/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8985 - auc_5: 0.9548 - loss: 0.2799 - val_accuracy: 0.9055 - val_auc_5: 0.0000e+00 - val_loss: 0.299
ch 48/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9006 - auc_5: 0.9610 - loss: 0.2637 - val_accuracy: 0.9140 - val_auc_5: 0.0000e+00 - val_loss: 0.297
ch 49/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8979 - auc_5: 0.9591 - loss: 0.2749 - val_accuracy: 0.9041 - val_auc_5: 0.0000e+00 - val_loss: 0.319
ch 50/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9036 - auc_5: 0.9617 - loss: 0.2654 - val_accuracy: 0.9083 - val_auc_5: 0.0000e+00 - val_loss: 0.335
ch 51/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8992 - auc_5: 0.9569 - loss: 0.2784 - val_accuracy: 0.9097 - val_auc_5: 0.0000e+00 - val_loss: 0.296
ch 52/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8969 - auc_5: 0.9581 - loss: 0.2735 - val_accuracy: 0.9055 - val_auc_5: 0.0000e+00 - val_loss: 0.329
ch 53/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9007 - auc_5: 0.9569 - loss: 0.2756 - val_accuracy: 0.8999 - val_auc_5: 0.0000e+00 - val_loss: 0.319
ch 54/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8816 - auc_5: 0.9533 - loss: 0.2882 - val_accuracy: 0.9210 - val_auc_5: 0.0000e+00 - val_loss: 0.304
ch 55/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8929 - auc_5: 0.9546 - loss: 0.2796 - val_accuracy: 0.9182 - val_auc_5: 0.0000e+00 - val_loss: 0.297
ch 56/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9023 - auc_5: 0.9561 - loss: 0.2761 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.305
ch 57/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8940 - auc_5: 0.9567 - loss: 0.2861 - val_accuracy: 0.9210 - val_auc_5: 0.0000e+00 - val_loss: 0.285
ch 58/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8926 - auc_5: 0.9624 - loss: 0.2608 - val_accuracy: 0.9252 - val_auc_5: 0.0000e+00 - val_loss: 0.281
```

```
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8926 - auc_5: 0.9624 - loss: 0.2608 - val_accuracy: 0.9252 - val_auc_5: 0.0000e+00 - val_loss: 0.281
ch 59/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8951 - auc_5: 0.9581 - loss: 0.2748 - val_accuracy: 0.9111 - val_auc_5: 0.0000e+00 - val_loss: 0.314
ch 60/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9154 - auc_5: 0.9673 - loss: 0.2431 - val_accuracy: 0.9252 - val_auc_5: 0.0000e+00 - val_loss: 0.318
ch 61/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9041 - auc_5: 0.9606 - loss: 0.2617 - val_accuracy: 0.9041 - val_auc_5: 0.0000e+00 - val_loss: 0.327
ch 62/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8943 - auc_5: 0.9586 - loss: 0.2755 - val_accuracy: 0.9055 - val_auc_5: 0.0000e+00 - val_loss: 0.31
ch 63/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9001 - auc_5: 0.9606 - loss: 0.2641 - val_accuracy: 0.9210 - val_auc_5: 0.0000e+00 - val_loss: 0.29
ch 64/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.8891 - auc_5: 0.9518 - loss: 0.2902 - val_accuracy: 0.9224 - val_auc_5: 0.0000e+00 - val_loss: 0.28
ch 65/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.9105 - auc_5: 0.9625 - loss: 0.2608 - val_accuracy: 0.9154 - val_auc_5: 0.0000e+00 - val_loss: 0.29
ch 66/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9063 - auc_5: 0.9607 - loss: 0.2682 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.314
ch 67/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9147 - auc_5: 0.9665 - loss: 0.2492 - val_accuracy: 0.9154 - val_auc_5: 0.0000e+00 - val_loss: 0.312
ch 68/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9210 - auc_5: 0.9687 - loss: 0.2369 - val_accuracy: 0.9140 - val_auc_5: 0.0000e+00 - val_loss: 0.315
ch 69/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9071 - auc_5: 0.9676 - loss: 0.2416 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.318
ch 70/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9039 - auc_5: 0.9591 - loss: 0.2736 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.316
ch 71/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9124 - auc_5: 0.9710 - loss: 0.2370 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.321
ch 72/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9047 - auc_5: 0.9608 - loss: 0.2626 - val_accuracy: 0.9097 - val_auc_5: 0.0000e+00 - val_loss: 0.318
ch 73/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9218 - auc_5: 0.9666 - loss: 0.2418 - val_accuracy: 0.9097 - val_auc_5: 0.0000e+00 - val_loss: 0.328
ch 74/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9006 - auc_5: 0.9599 - loss: 0.2715 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.325
ch 75/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9099 - auc_5: 0.9641 - loss: 0.2548 - val_accuracy: 0.9154 - val_auc_5: 0.0000e+00 - val_loss: 0.316
ch 76/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8979 - auc_5: 0.9577 - loss: 0.2688 - val_accuracy: 0.9154 - val_auc_5: 0.0000e+00 - val_loss: 0.313
ch 77/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9189 - auc_5: 0.9674 - loss: 0.2415 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.330
ch 78/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9082 - auc_5: 0.9692 - loss: 0.2320 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.327
ch 79/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9011 - auc_5: 0.9628 - loss: 0.2599 - val_accuracy: 0.9013 - val_auc_5: 0.0000e+00 - val_loss: 0.337
ch 80/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8955 - auc_5: 0.9595 - loss: 0.2731 - val_accuracy: 0.9182 - val_auc_5: 0.0000e+00 - val_loss: 0.315
ch 81/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8988 - auc_5: 0.9548 - loss: 0.2798 - val_accuracy: 0.9027 - val_auc_5: 0.0000e+00 - val_loss: 0.330
ch 82/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9144 - auc_5: 0.9653 - loss: 0.2443 - val_accuracy: 0.9252 - val_auc_5: 0.0000e+00 - val_loss: 0.303
ch 83/100
```

```
ch 83/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9174 - auc_5: 0.9713 - loss: 0.2230 - val_accuracy: 0.9154 - val_auc_5: 0.0000e+00 - val_loss: 0.312
ch 84/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9091 - auc_5: 0.9629 - loss: 0.2626 - val_accuracy: 0.9168 - val_auc_5: 0.0000e+00 - val_loss: 0.315
ch 85/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9174 - auc_5: 0.9683 - loss: 0.2404 - val_accuracy: 0.9238 - val_auc_5: 0.0000e+00 - val_loss: 0.30
ch 86/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9053 - auc_5: 0.9684 - loss: 0.2422 - val_accuracy: 0.9083 - val_auc_5: 0.0000e+00 - val_loss: 0.31
ch 87/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.9110 - auc_5: 0.9724 - loss: 0.2268 - val_accuracy: 0.9154 - val_auc_5: 0.0000e+00 - val_loss: 0.31
ch 88/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9154 - auc_5: 0.9719 - loss: 0.2242 - val_accuracy: 0.9041 - val_auc_5: 0.0000e+00 - val_loss: 0.330
ch 89/100
52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9107 - auc_5: 0.9661 - loss: 0.2456 - val_accuracy: 0.9013 - val_auc_5: 0.0000e+00 - val_loss: 0.327
ch 90/100
52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9102 - auc_5: 0.9670 - loss: 0.2418 - val_accuracy: 0.9126 - val_auc_5: 0.0000e+00 - val_loss: 0.306
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
- Loss: 0.3055
- AUC: 0.9460
- Accuracy: 0.9010
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.90      0.90      0.90       394
           1       0.90      0.90      0.90       394

    accuracy                           0.90       788
   macro avg       0.90      0.90      0.90       788
weighted avg       0.90      0.90      0.90       788
```

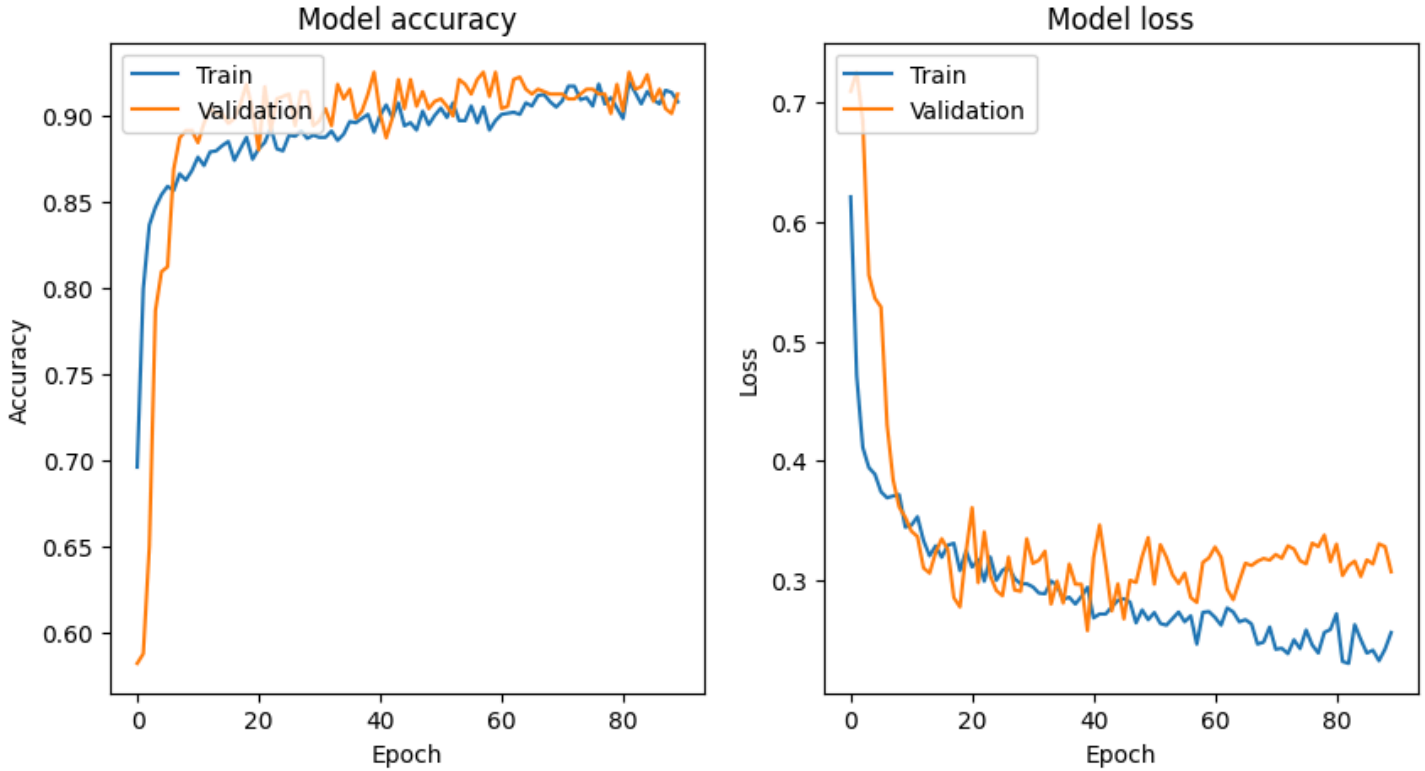
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
```

```
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

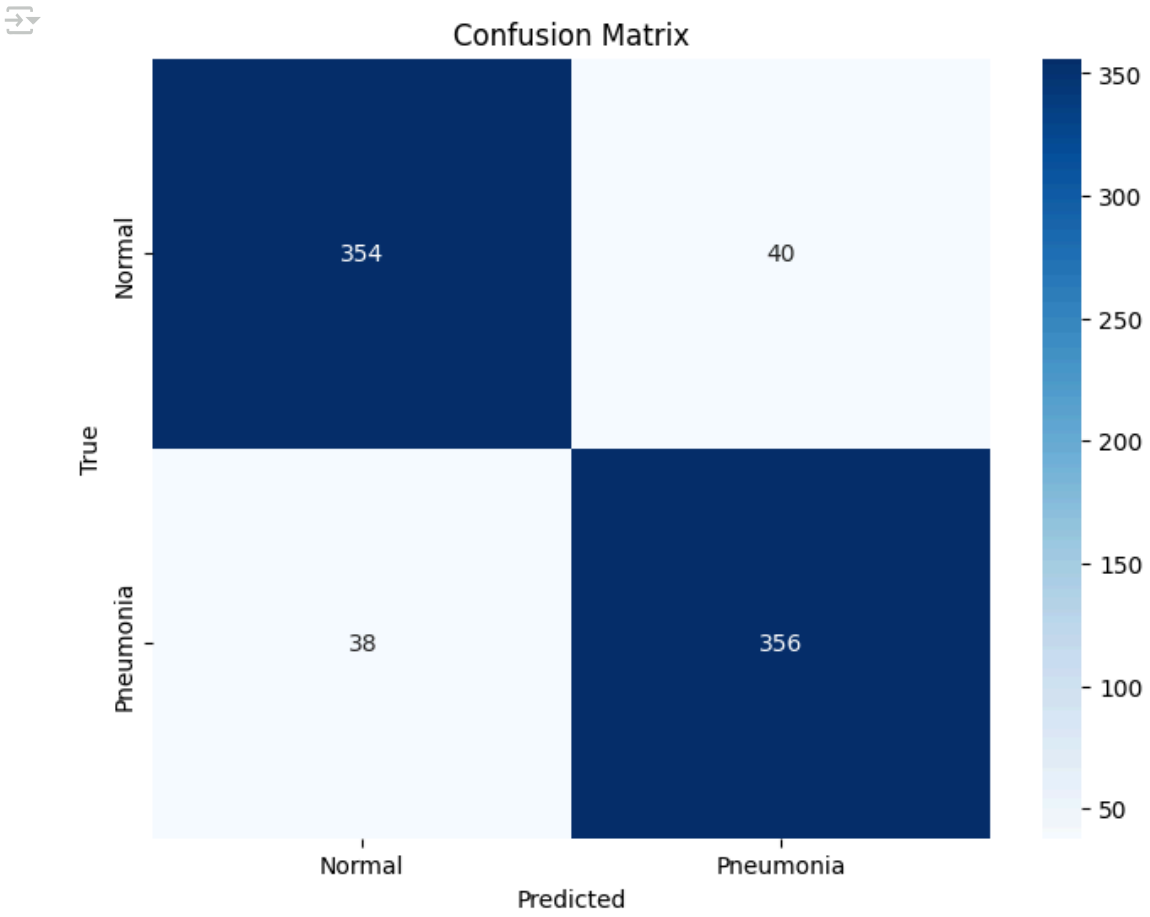
```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```



CODE END HERE

```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/NORMAL'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)
```

```
for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
        axes[i].set_title(f"Prediction: {prediction_for_sample}")
        axes[i].axis('off')

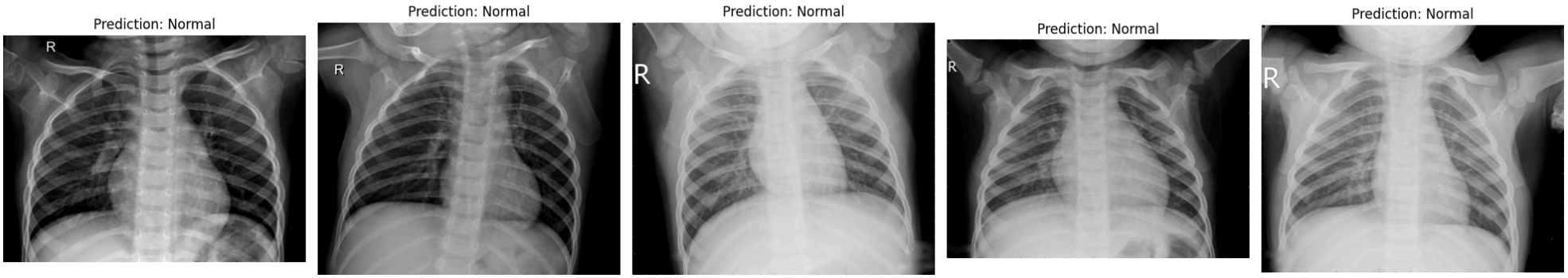
    except Exception as e:
        print(f"Error processing {image_file}: {e}")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
```

	Image Path	Prediction
0	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
1	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
2	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
3	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
4	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
5	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
6	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal
7	/content/drive/MyDrive/TA/Dataset/val/NORMAL/N...	Normal

Sample Predictions



OTHER EXPERIMENT

```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/PNEUMONIA'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
```